

DeepEval Mastery: The 10-Concept Enterprise Reference

Comprehensive Architecture Cheat Sheet for AI Agent & Reliability Engineers

Concept 1: The LLMTestCase Primitive & Assertion Mechanics

LLMTestCase encapsulates test data: **input** (user query), **actual_output** (LLM response), **retrieval_context** (RAG chunks), **expected_output** (golden truth), and **tools_called** / **expected_tools** (agent tool execution traces). **assert_test()** halts immediately with **AssertionError** (exit code 1) on metric failure for CI gates. **evaluate()** runs across entire datasets without throwing exceptions, outputting macro telemetry.

Concept 2: The RAG Triad (Generation vs. Retrieval Metrics)

Decouples evaluation between the **Retriever (Shopper)** and the **Generator (Chef)**:

- **FaithfulnessMetric**: Measures hallucination (**actual_output** verified strictly against **retrieval_context**). Ground truth = **retrieval_context**.
- **AnswerRelevancyMetric**: Evaluates if **actual_output** addresses input without topic drift.
- **ContextualRecallMetric**: Measures retrieval completeness (facts in **expected_output** found in **retrieval_context**). Completely ignores **actual_output**.
- **ContextualPrecisionMetric**: Evaluates retrieval ranking. Penalizes irrelevant chunks outranking relevant ones, mitigating the 'Lost in the Middle' attention degradation effect.

Concept 3: The G-Eval Framework (Custom LLM-as-a-Judge)

Replaces arbitrary integer (1-5) prompting with a two-phase engine: **Auto-CoT Step Generation** (synthesizing structured evaluation steps from criteria) and **Probability Normalization** (measuring score token log probabilities to calculate expected value: $\text{Score} = \sum i \times P(\text{Score} = i)$). Drastically eliminates integer scoring bias and flakiness.

Concept 4: Synthetic Data Generation & Evolution (Synthesizer)

Solves golden dataset cold-starts by parsing raw knowledge documents and mutating seed prompts into multi-hop queries, reasoning tasks, and constrained edge-case tests, guaranteeing grounded reference test suites.

Concept 5: Agentic Evaluation & Tool-Call Verification

Audits intermediate agent tool trajectories using **ToolCorrectnessMetric** and **ToolCallParams**. Validates tool selection and parameter schema adherence, preventing end-to-end output masking from hiding internal execution regressions.

Concept 6: Automated Red Teaming & Vulnerability Scanning

Automates adversarial attack probes: Direct/Indirect Prompt Injection, PII leakage, toxicity, and jailbreaks. Uses attack enhancements (Base64 obfuscation, persona simulation) to verify application guardrails before release.

Concept 7: Deterministic Unit Guardrails & Custom BaseMetric

Subclassing **BaseMetric** enables zero-token, low-latency (<5ms) Python assertions (regex validation, JSON schema checks, token count limits, PII filters) without invoking an expensive LLM judge.

Concept 8: Production Observability, Live Tracing & Continuous Eval Flywheel

Avoids 100% judge evaluation costs on live traffic through a tiered architecture: 100% deterministic heuristic filters, 2%–5% asynchronous reference-free judge sampling, and 100% user-downvoted trace capture. Anomalous traces are anonymized, converted to deterministic LLMTestCase fixtures, and added to the CI regression suite.

Concept 9: Model Benchmarking, Evaluation Datasets & Trade-Offs

Leverages **EvaluationDataset** to benchmark model updates across the enterprise Pareto frontier: measuring accuracy (Faithfulness / GEval) against p95 latency and token cost per 1,000 requests.

Concept 10: Multi-Turn Conversational Evaluation & Memory Drift

Uses **ConversationalTestCase(turns=[...])** to audit dialogue trajectories turn-by-turn. Prevents single-turn string concatenation from hiding failures, identifies Dialogue Memory Drift, and audits conversational query rewriters resolving anaphora (pronoun references).

Enterprise CI/CD Pytest Implementation Pattern

```
# tests/evals/test_agent_reliability.py
import pytest
from deepeval import assert_test
from deepeval.test_case import LLMTestCase, ToolCallParams
from deepeval.metrics import FaithfulnessMetric, ToolCorrectnessMetric

@pytest.fixture
def agent_trace():
    return LLMTestCase(
        input="Query refund window for Enterprise Tier.",
        actual_output="Enterprise licenses have a 45-day refund window.",
        retrieval_context=["Enterprise licenses have a 45-day refund window."],
        tools_called=[ToolCallParams(name="fetch_policy", input_parameters={"tier": "enterprise"})],
        expected_tools=[ToolCallParams(name="fetch_policy", input_parameters={"tier": "enterprise"})]
    )

def test_agent_regression(agent_trace):
    faithfulness = FaithfulnessMetric(threshold=0.8, include_reason=True)
    tool_check = ToolCorrectnessMetric()
    assert_test(agent_trace, [faithfulness, tool_check])
```

Master Architecture & Metric Decision Matrix

Metric / Component	Target Subsystem	Required Fields	Ref Needed?	Primary Failure Mode Detected
FaithfulnessMetric	Generator (LLM)	actual_output, retrieval_context	No	Hallucinations, ungrounded claims, factual contradictions
AnswerRelevancyMetric	Generator (LLM)	input, actual_output	No	Incomplete answers, topic drift, evasive responses
ContextualRecallMetric	Retriever (Vector DB)	expected_output, retrieval_context	Yes (Golden)	Retriever omitted key factual chunks from context
ContextualPrecisionMetric	Retriever / Re-ranker	input, expected_output, retrieval_context	Yes (Golden)	Irrelevant chunks ranked above relevant ones
ToolCorrectnessMetric	Agent Tool Execution	tools_called, expected_tools	Yes (Tools)	Invalid tool call, schema divergence, skipped actions
GEval	Custom / Domain Logic	Declared in evaluation_params	Configurable	Policy violations, tone drift, custom business rule breaks
ConversationalTestCase	Multi-Turn Dialogue	Ordered list of LLMTTestCase turns	Per-turn	Dialogue memory drift, pronoun/anaphora resolution bugs
BaseMetric (Custom)	Deterministic Heuristics	Programmatic Python properties	Optional	Malformed JSON, regex mismatch, PII leakage, latency caps